

K-Nearest Neighbors

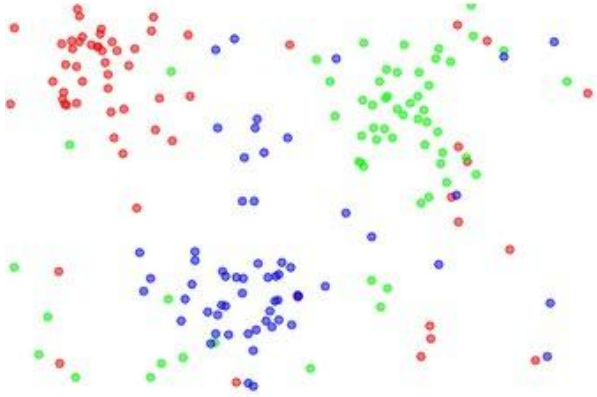
Hrachya Asatryan

K-Nearest Neighbors (KNN)

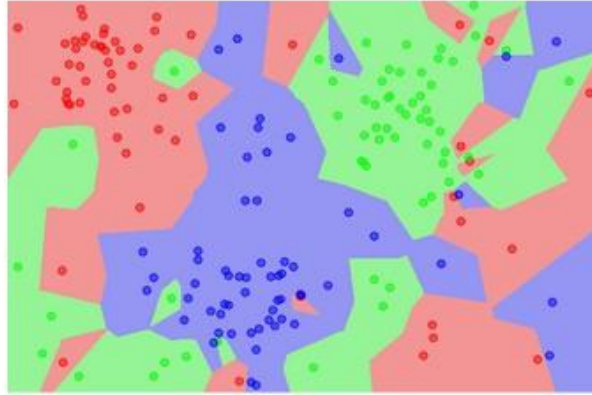
- **The Intuition:** "Tell me who your friends are, and I'll tell you who you are."
- **The Logic:**
 - We assume similar things exist in close proximity.
 - If a point is sitting in a sea of "Blue" points, it is probably "Blue."
- **Type of Learning:**
 - **Supervised:** Requires labeled data.
 - **Instance-Based (Lazy):** No training phase! It just memorizes the data.

K-NN decision regions

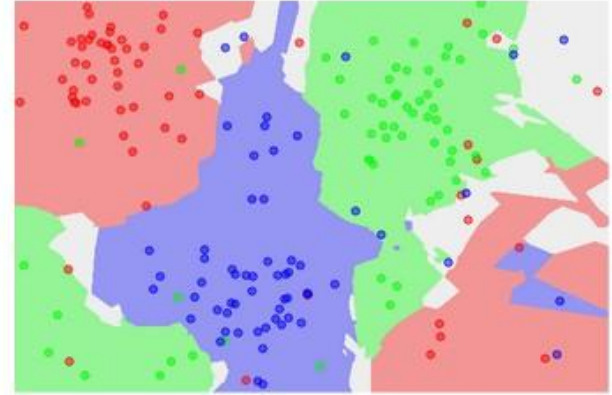
the data



NN classifier



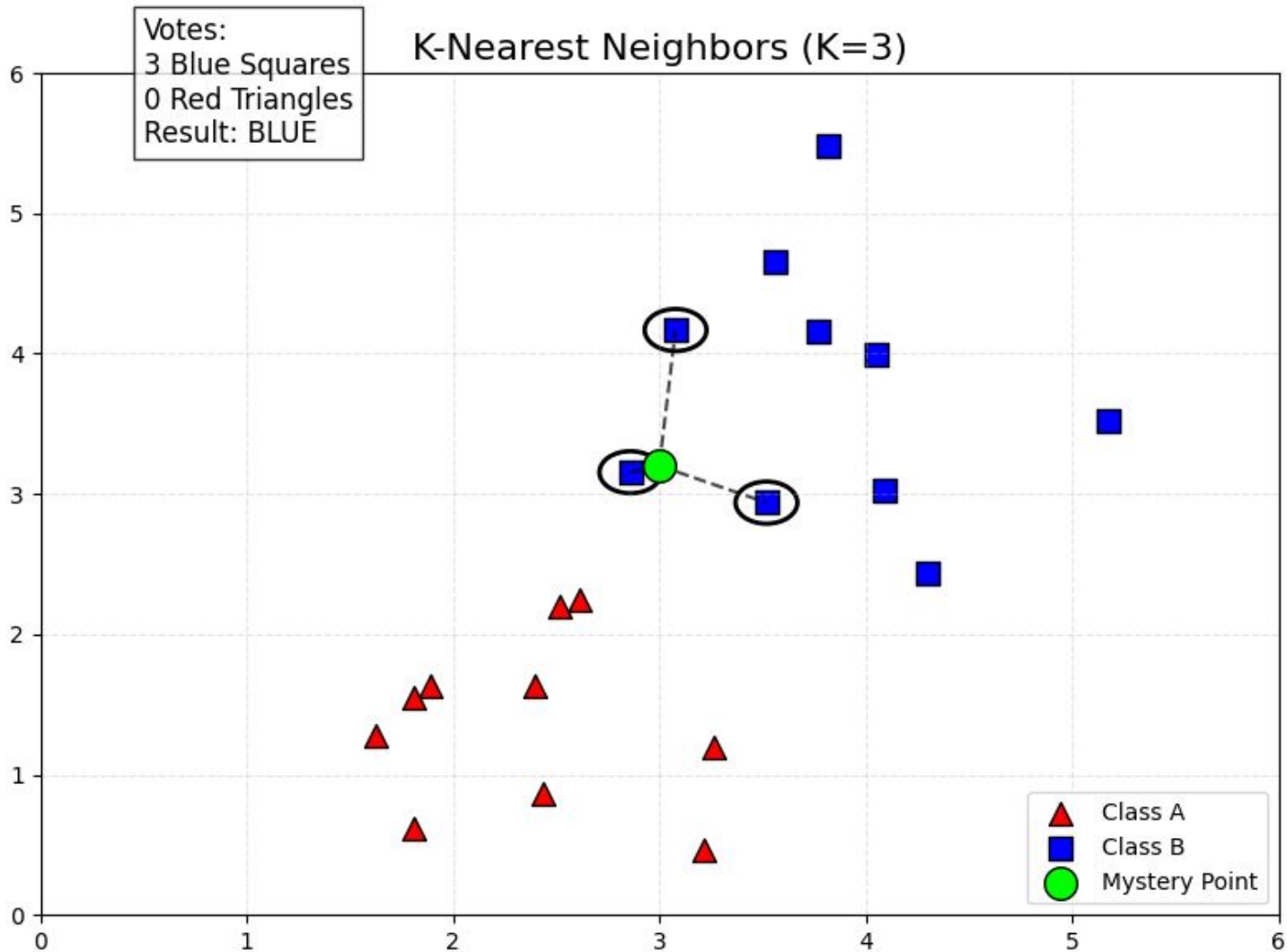
5-NN classifier



VERY IMPORTANT PREPROCESSING STEP: SCALING!!!!!!
if we don't scale, this model becomes meaningless

The Algorithm (Step-by-Step)

- **Input:** Dataset (x_k, y_k) and a new observation x .
- **Step 1:** Choose a number K (e.g., 3).
- **Step 2:** Choose a **Distance Metric** (How do we measure "Close"?).
- **Step 3:** Find the K points in the data closest to x .
- **Step 4 (Classification):**
 - **Vote:** Count the labels of the K neighbors.
 - **Decision:** The majority wins. (Tie-breaker: Pick randomly or weight by distance).



Mathematical definition

Our aim is to find the maximum of

$$P(Y = 0|X = x) \text{ and } P(Y = 1|X = x).$$

What is the probability that the result (Y) is 0 (or 1), given the input (x)?

In this case, we approximate the Probabilities in the following way: first we take a natural number k and define $NN_k(x)$ = The set of k Nearest Points x_i from x . Then we take

$$P(Y = 0|X = x) \approx \#\{y_i = 0 \text{ and } x_i \in NN_k(x)\} / k,$$

and, similarly,

Numerator: The number of neighbors that belong to Class 0.

Denominator: The total number of neighbors (k).

The Logic: If 4 out of 5 neighbors ($k=5$) are Class 0, the probability is $4/5$ or 80%.

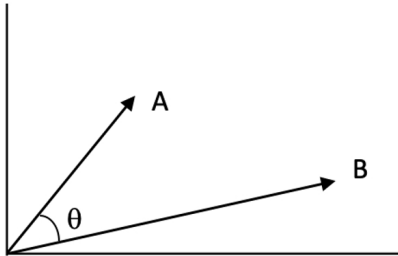
$$P(Y = 1|X = x) \approx \#\{y_i = 1 \text{ and } x_i \in NN_k(x)\} / k,$$

Mathematical definition

Now, to compare the Probabilities of labels, it is enough to compare the numerators

$$\#\{y_i = 0 \text{ and } x_i \in \text{NN}_k(x)\} \text{ and } \#\{y_i = 1 \text{ and } x_i \in \text{NN}_k(x)\}$$

And we will predict that label, for which the above number is maximal. This is the idea of the k-NN. Of course, we need to define a distance to calculate the Nearest Points to x .



$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Visualization

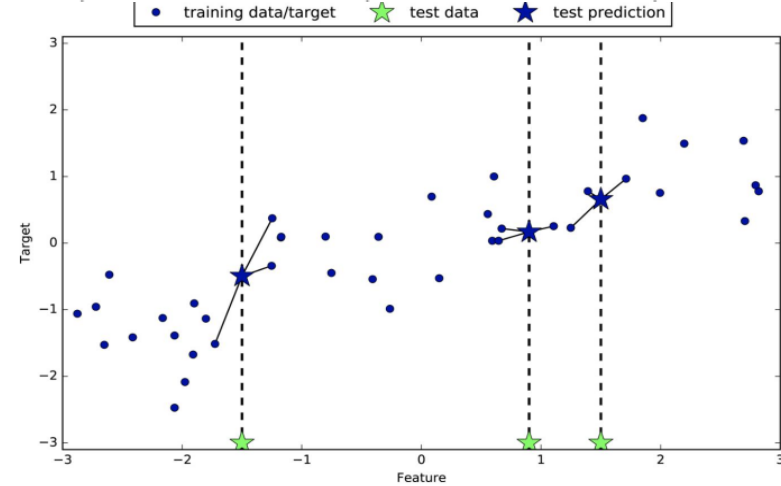
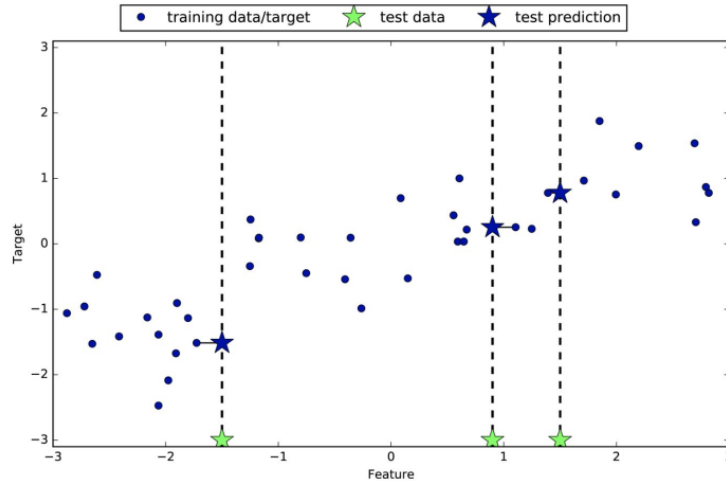
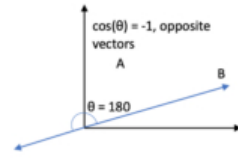
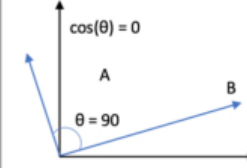
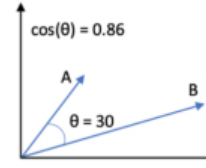
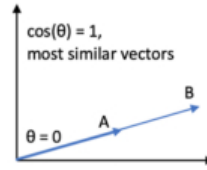


figure 2 . predictions make by three- nearest-neighbors regression on the wave dataset

Figure taken from:

<https://medium.com/analytics-vidhya/k-neighbors-regression-analysis-in-python-61532d56d8e4>

Measuring Similarity: Distance Metrics

Euclidean Distance (L2 Norm):

- The "Ruler" distance. Straight line.
- *NumPy*: `np.linalg.norm(a - b)`

Manhattan Distance (L1 Norm):

- The "Taxi" distance. Grid movement only.
- *Use case*: High dimensional sparse data.

NOTE: There are many different distance metrics for each specific type of data.

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

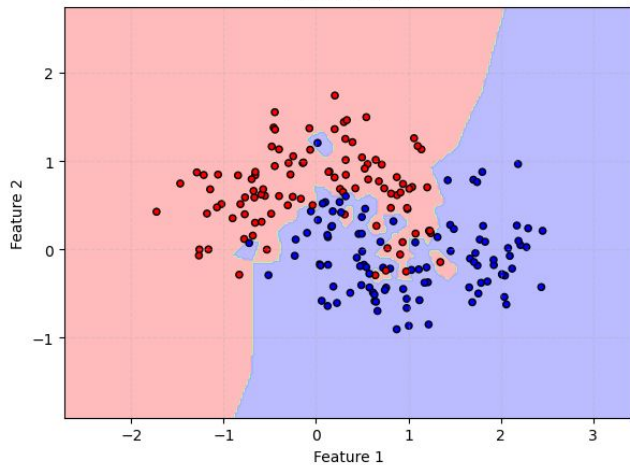


The "K" in KNN (Hyperparameters)

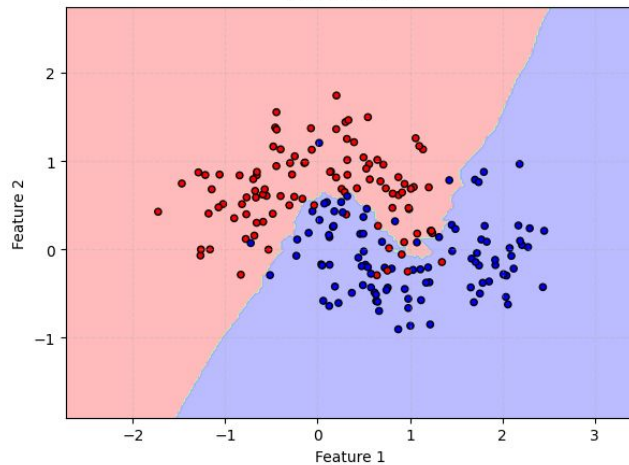
- **What is a Hyperparameter?** A setting we choose before the model runs (unlike weights, which the model learns).
- **Case 1: K=1 (The Over-Reactor)**
 - The decision boundary is jagged and chaotic.
 - Every outlier forces the boundary to curve around it.
 - Captures Noise.
- **Case 2: K=100 (The Conformist)**
 - The boundary is smooth and simple.
 - Subtle patterns are ignored.
 - Misses the point.

K-NN decision regions

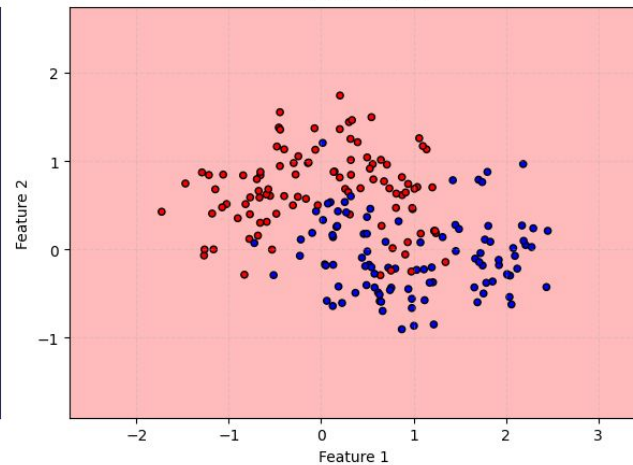
K=1 (Overfitting)
'The Paranoid'



K=15 (Balanced)
'The Rational'



K=N (Underfitting)
'The Conformist'



KNN is not just for Classification

- **KNN Regression:**

- Instead of voting (Red vs Blue), we **Average**.
- If neighbors have values `[10, 12, 14]`, prediction is `12`.

- **The Effect of K on Regression:**

- Small K: The prediction line looks shaky (follows every bump).
- Large K: The prediction line looks flat (averages everything out).

KNN Regression example

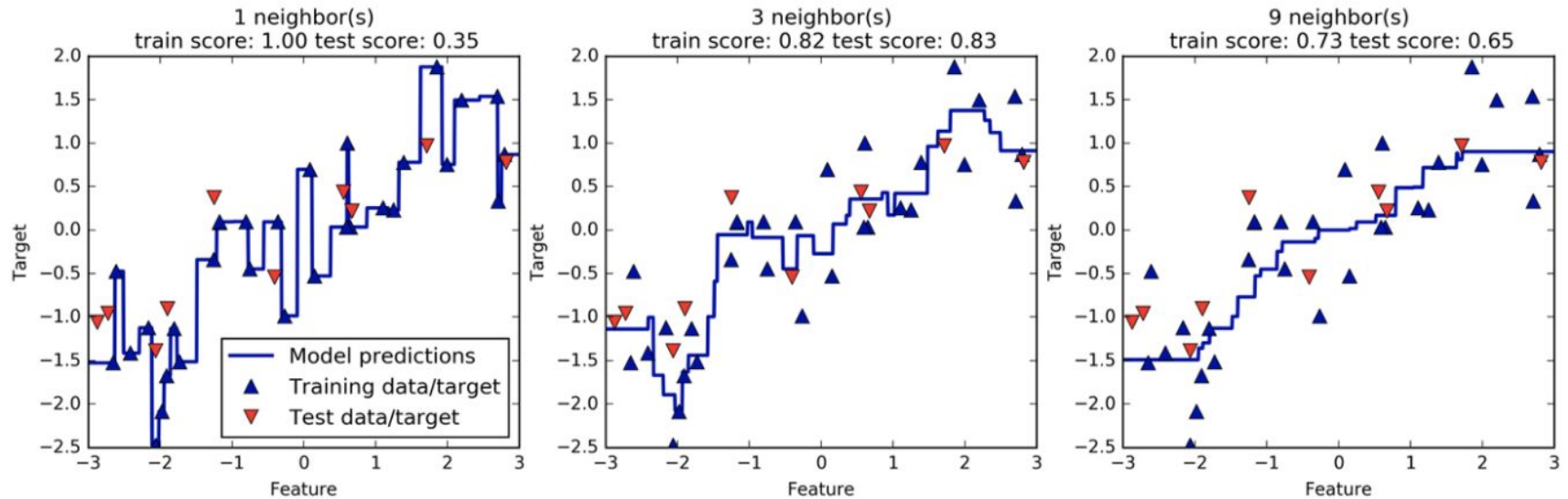


Figure taken from:

<https://medium.com/analytics-vidhya/k-neighbors-regression-analysis-in-python-61532d56d8e4>

The Pros and Cons

Pros:

- Simple to understand.
- No training time ($O(1)$ training).
- Non-parametric (doesn't assume data is a straight line).

Cons:

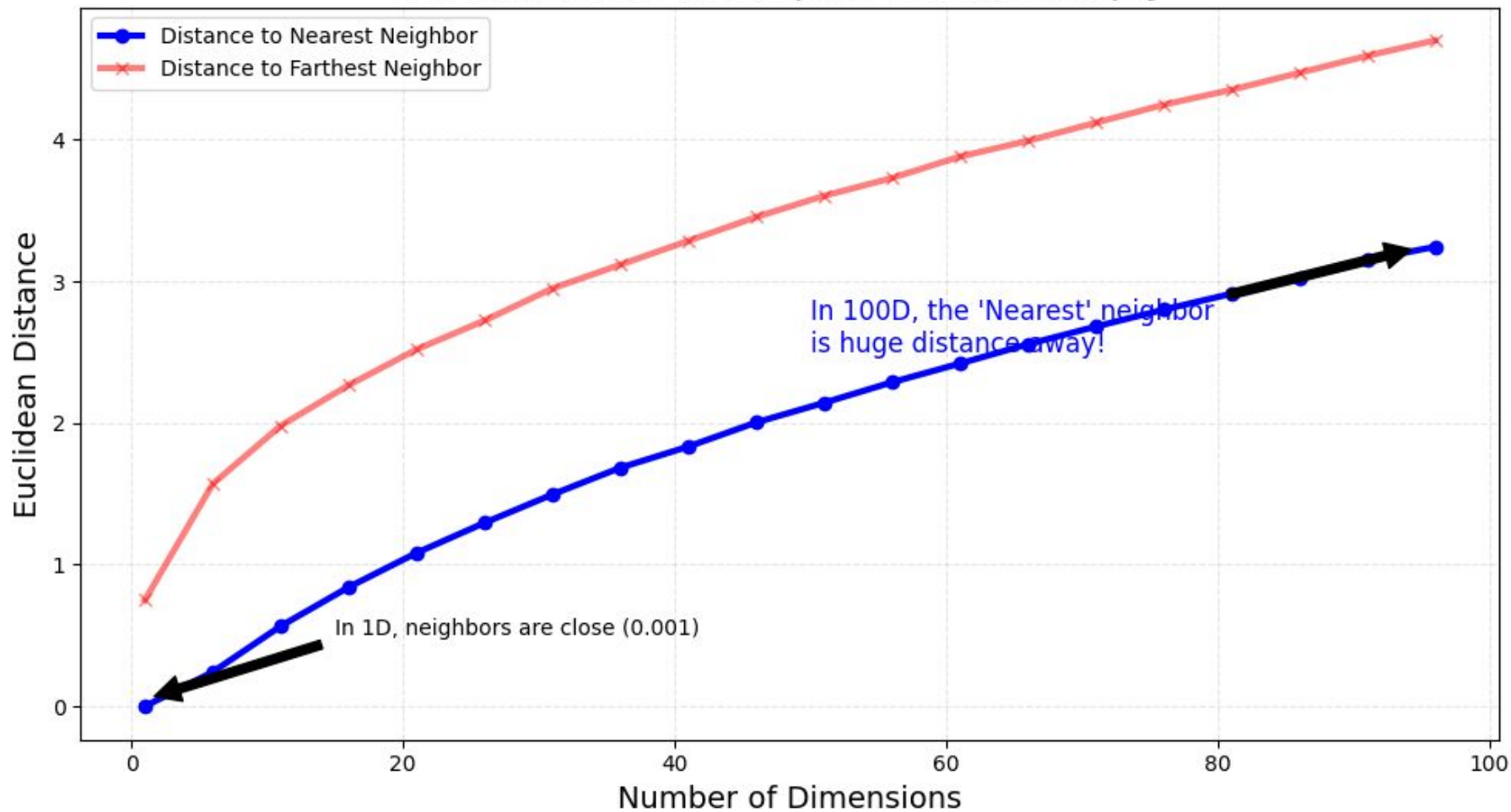
- **Slow Inference:** To predict *one* point, you must calculate distance to *every* point. ($O(N)$ prediction).
- Memory Intensive: Must keep all data in RAM.
- **The Curse of Dimensionality.**

The Curse of Dimensionality

- **The Problem:** In high dimensions (e.g., 1000 columns), *all* points are far away from each other.
- **The Geometry:**
 - In 2D, a circle fills most of a square.
 - In 100D, the "volume" is all in the corners.
 - "Nearest" neighbor becomes meaningless because everyone is equally distant.
- **Impact:** KNN breaks down with too many features (like raw pixels).

The Curse of Dimensionality

As Dimensions Rise, Space Becomes Empty



$$w_i = \frac{1}{d(y_i, y_j) + \epsilon}$$

Weight of neighbor i
 (closer = higher weight)

Distance between
 target & neighbor
 (e.g., Cosine Distance)

Small constant to prevent
 division by zero

$$\text{Final Weighted Average Prediction} \rightarrow \frac{\sum_{i=1}^k w_i X}{\sum_{i=1}^k w_i}$$

Neighbor's value (X) weighted
 by its importance (w_i)

Sum of all weights
 (for normalization)